



Actividades prácticas: Arreglos y encapsulamiento en Java

Objetivos

- Diseñar e implementar algoritmos eficientes y estructurados para modificar y procesar arreglos encapsulados en una clase.
- Verificar los servicios implementados para un conjunto significativo de casos de prueba.

EJERCICIO 1.

- Definí el concepto de encapsulamiento y describí los recursos que brinda Java para encapsular.
- ¿Cuáles son las ventajas del encapsulamiento? ¿Cuál sería el problema si no encapsulamos los atributos de instancia de una clase?
- Definí el concepto de Tipo de Dato Abstracto.

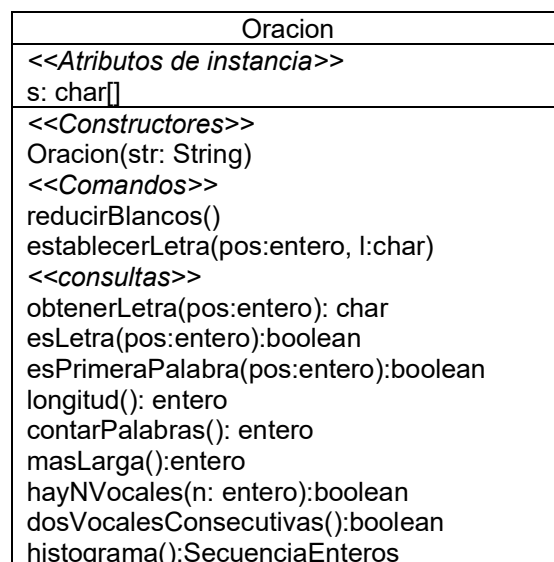
EJERCICIO 2. Implemente y verifique la clase *SecuenciaEnteros* definida como sigue:

SecuenciaEnteros
<<Atributos de instancia>> sec: entero []
<<Constructor>> SecuenciaEnteros(cant:entero) <<Comandos>> establecerEntero(p,n: entero) reemplazar(n1,n2:entero) reemplazar(n:entero) intercambiar(p1,p2:entero):boolean copy(a:SecuenciaEnteros):boolean <<Consultas>> obtenerEntero(p:entero):entero cantElementos():entero total():entero estaNum(n:entero): boolean cantidadMayores(n:entero):entero mitadMayores(n:entero):boolean equals(a:SecuenciaEnteros):boolean clone():SecuenciaEnteros

- *SecuenciaEnteros(cant: entero)*. Requiere $\text{cant} > 10$
- *establecerEntero(p,n:entero)*. Requiere $0 \leq p < \text{cantElementos}()$.
- *reemplazar(n1,n2:entero)*. Reemplaza toda aparición de n1 por n2 en sec.
- *reemplazar(n:entero)*. Reemplaza la primera y la última aparición de n por 0 en sec. Si solo hay una aparición de n en sec, la reemplaza por 0 y si no hay ninguna no provoca ningún efecto.
- *intercambiar(p1,p2:entero):boolean*. Si $0 \leq p1, p2 < \text{cantElementos}()$ intercambia los elementos que ocupan las posiciones p1 y p2 en sec y retorna true, sino retorna false.
- *copy(a:SecuenciaEnteros):boolean*. Si el parámetro a está ligado y tiene la misma cantidad de elementos que el objeto que recibe el mensaje, retorna true y copia cada elemento de la secuencia a en el objeto que recibe el mensaje, manteniendo la misma posición; sino retorna false.
- *obtenerEntero(p:entero):entero*. Requiere $0 \leq p < \text{cantElementos}()$.

- *cantElementos(): entero*. Retorna el tamaño de sec.
- *total(): entero*. Retorna la sumatoria de todos los elementos de sec.
- *estaNum(n:entero): boolean*. Retorna true si y solo sí al menos un número en la secuencia es igual a n.
- *cantidadMayores(n:entero):entero*. Retorna la cantidad de números mayores a n.
- *mitadMayores(n:entero):boolean*. Retorna true si al menos la mitad de los números son mayores a n.

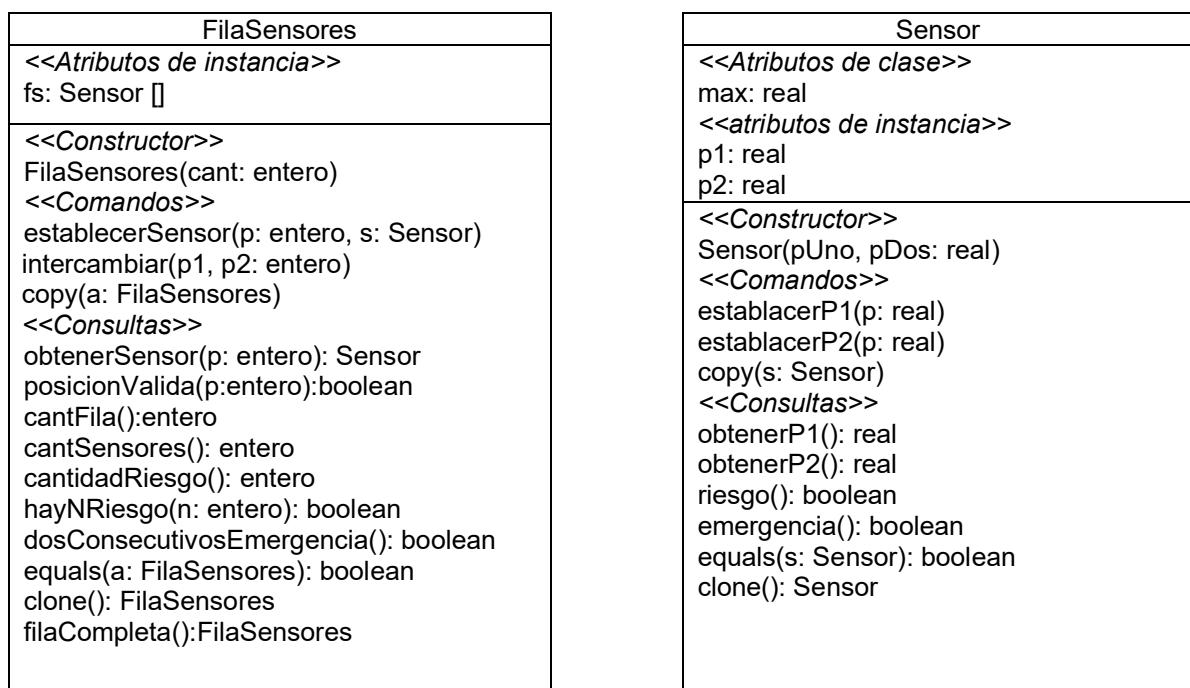
EJERCICIO 3. Implemente y verifique la clase *Oracion* modelada en el siguiente diagrama:



- *Oracion(str: String)*. Requiere que:
 - la cadena *stresté* formada por una o más palabras separadas por al menos un blanco
 - comienza con una palabra y termina con un blanco
 - cada palabra de la oración esté formada por una o más letras mayúsculas
- *obtenerLetra(pos:entero):charyestablecerLetra(pos:entero, l:char)*. Requieren que la posición posea válida y l sea una letra mayúscula o un espacio en blanco.
- *reducirBlancos()*. Reemplaza las secuencias de dos o más blancos por un solo blanco, excepto al final de la oración. La oración
 "HOY VA A LLOVER A LA TARDE " se transforma en
 "HOY VA A LLOVER A LA TARDE "
- *esLetra(pos:entero):boolean*. Retorna verdadero siempre que pos sea una posición válida y en esa posición se haya asignado una letra, sino retorna falso.
- *esPrimeraPalabra(pos:entero):boolean*. Retorna verdadero siempre que pos sea una posición válida, en esa posición se haya asignado una letra y sea la primera de una palabra.
- *longitud():entero*. Retorna el tamaño del arreglo

- *contarPalabras(): entero*. Retorna la cantidad de palabras de la oración.
- *masLarga(): entero*. Retorna la longitud de la palabra más larga.
- *hayNVocales(n: entero): boolean*. Retorna true si y solo sí la oración contiene exactamente n vocales.
- *dosVocalesConsecutivas(): boolean*. Retorna true si alguna palabra contiene dos vocales en posiciones consecutivas.
- *histograma(): SecuenciaEnteros*. Retorna un objeto de clase *SecuenciaEnteros* que registra la cantidad de apariciones de cada letra del alfabeto en la oración que recibe el mensaje. Observe que la cantidad de elementos de *SecuenciaEnteros* corresponde a la cantidad de letras del alfabeto.

EJERCICIO 4. Implemente y verifique el siguiente diagrama de clases considerando que desde la clase cliente las posiciones en la fila se referencian desde 1 hasta el tamaño de la fila. De modo que la **posición p en la fila corresponde al subíndice p-1 en el arreglo fs encapsulado en la clase**.



- *FilaSensores(cant: entero)*. Requiere cant > 0.
- *establecerSensor(p: entero, s: Sensor)*. Si la posición p es válida, asigna el sensor s al subíndice p-1 en el arreglo fs, en caso contrario no tiene efecto.
- *intercambiar(p1, p2: entero)*. Si p1 y p2 son posiciones válidas en la fila, intercambia en el arreglo fs las referencias que corresponden a los subíndices p1-1 y p2-1.
- *copy(a: FilaSensores)*. Implementa copy en profundidad. Requiere que el parámetro a esté ligado, las dos filas tengan el mismo tamaño y que en ambas filas cada referencia esté ligada a un sensor.

- *obtenerSensor(p: entero): Sensor*. Si la posición *p* es válida retorna el sensor asignado al subíndice *p-1*, en caso contrario retorna null.
- *posicionValida(p: entero): boolean*. Retorna verdadero si *p* es una posición válida en la fila.
- *cantFila(): entero*. Retorna el tamaño de la fila, es decir la cantidad de componentes del arreglo *fs*.
- *cantSensores(): entero*. Retorna la cantidad de sensores de la fila que recibe el mensaje, es decir la cantidad de componentes ligadas en el arreglo *fs*.
- *cantidadRiesgo(): entero*. Retorna la cantidad de sensores en riesgo. Requiere que todas las referencias en el arreglo estén ligadas.
- *hayNRiesgo(n: entero): boolean*. Retorna true si la fila contiene al menos *n* sensores en riesgo. Requiere que todas las referencias en el arreglo estén ligadas.
- *dosConsecutivosEmergencia(): boolean*. Retorna true si la estructura contiene dos sensores en riesgo en posiciones consecutivas. Requiere que todas las referencias en el arreglo estén ligadas.
- *equals(a: FilaSensores): boolean*. Implementa igualdad superficial. Requiere que el parámetro *a* esté ligado.
- *clone(): FilaSensores*. Implementa clone en profundidad. Requiere que todas las referencias del arreglo estén ligadas.
- *filaCompleta(): FilaSensores*. Retorna una nueva fila con los mismos sensores que la fila que recibe el mensaje, en el mismo orden (no necesariamente en las mismas posiciones), y con las referencias no ligadas al final de la estructura.

Implemente la clase *OtraFilaSensores* con el mismo diagrama, pero considerando que los siguientes servicios tienen una especificación diferente:

- *cantidadRiesgo(): entero*. Retorna la cantidad de sensores en riesgo.
- *hayNRiesgo(n: entero): boolean*. Retorna true si la fila contiene al menos *n* sensores en riesgo.
- *dosConsecutivosEmergencia(): boolean*. Retorna true si la estructura contiene dos sensores en riesgo en posiciones consecutivas.
- *equals(a: OtraFilaSensores): boolean*. Implementa igualdad superficial.
- *clone(): OtraFilaSensores*. Implementa clone superficial.

EJERCICIO 5. Implemente y verifique la clase *GrillaReales* definida como sigue:

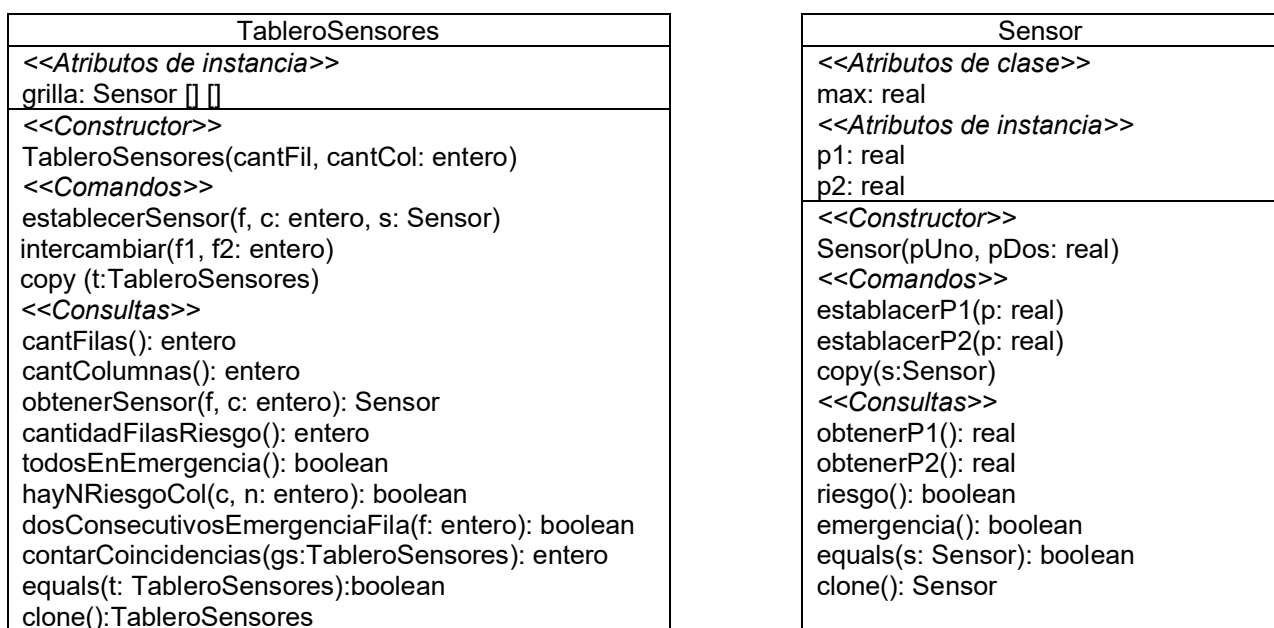
GrillaReales
<<Atributos de instancia>> grilla: float [][]
<<Constructor>> GrillaReales(f, c: entero)
<<Comandos>> establecerReal(f, c: entero, r: real) reemplazar(r1, r2: real) reemplazar(r1, r2: real, f: entero) intercambiarFilas(f1, f2: entero): boolean
<<Consultas>> obtenerReal(f, c: entero): real cantFilas(): entero cantColumnas(): entero total(): real totalColumna(c: entero): real estaNum(r: real): boolean cantidadMayores(r: real): entero esFilaCreciente(f: entero): boolean hayNMayoresConsecutivos(n: entero, r: real): boolean esTranspuesta(g: GrillaReales): boolean transpuesta(): GrillaReales

- *GrillaReales(f, c: entero)*. Requiere $f, c > 0$.
- *establecerReal(f, c: entero, r: real)*. Requiere $0 \leq f < \text{cantFilas}()$ y $0 \leq c < \text{cantColumnas}()$.
- *reemplazar(r1, r2: real)*. Reemplaza toda aparición de r1 por r2 en la grilla.
- *reemplazar(r1, r2: real, f: entero)*. Reemplaza la última aparición de r1 por r2 en la fila f. Requiere $0 \leq f < \text{cantFilas}()$.
- *intercambiarFilas(f1, f2: entero): boolean*. Si $0 \leq f1, f2 < \text{cantFilas}()$ intercambia los elementos de las filas f1 y f2 y retorna true, sino retorna false.
- *obtenerReal(f, c: entero): real*. Requiere $0 \leq f < \text{cantFilas}()$ y $0 \leq c < \text{cantColumnas}()$.
- *total(): real*. Retorna la suma de todos los elementos de la grilla.
- *totalColumna(c: entero): real*. Requiere $0 \leq c < \text{cantColumnas}()$ computa la suma de los elementos de la columna c.
- *estaNum(r: real): boolean*. Retorna true si al menos un elemento de la grilla es igual a r.
- *cantidadMayores(r: real): entero*. Retorna la cantidad de números mayores a r en la grilla.
- *esFilaCreciente(f: entero): boolean*. Requiere $0 \leq f < \text{cantFilas}()$. Retorna true si los elementos de la fila f están ordenados en forma estrictamente creciente.
- *hayNMayoresConsecutivos(n: entero, r: real): boolean*. Retorna true si al menos una fila de la grilla contiene exactamente n elementos consecutivos mayores a r.
- *esTranspuesta(g: GrillaReales): boolean*. Retorna true si y solo si g es la matriz transpuesta de la matriz que recibe el mensaje.
- *transpuesta(): GrillaReales*. Retorna la transpuesta de la matriz que recibe el mensaje.

Analice cómo modificaría la implementación si la especificación fuera:

- *totalColumna(c: entero): real*. Computa la suma de los elementos de la columna c. Retorna -1 si c no es válido.
- *hayNMayoresConsecutivos(n: entero, r: real): boolean*. Retorna true si exactamente una fila de la grilla contiene al menos n elementos consecutivos mayores a r.

EJERCICIO 6. Implemente y verifique el siguiente diagrama de clases:



- *TableroSensores(cantFil, cantCol: entero)*. Requiere cantFil, cantCol > 0.
- *establecerSensor(f, c: entero, s: Sensor)*. Requiere 0 <= f < cantFilas() y 0 <= c < cantColumnas().
- *intercambiar(f1, f2: entero)*. Requiere 1 <= f1, f2 <= cantFilas(). Intercambia los sensores de las filas f1 y f2.
- *copy(t: TableroSensores)*. Si el parámetro t está ligado y el tablero que recibe el mensaje y el que pasa como parámetro tienen la misma cantidad de filas y columnas, implementa copy superficial, en caso contrario no tiene ningún efecto.
- *obtenerSensor(f, c: entero): Sensor*. Requiere 0 <= f < cantFilas() y 0 <= c < cantColumnas().
- *cantidadFilasRiesgo(): entero*. Retorna la cantidad de filas que contienen al menos un sensor en riesgo.
- *todosEnEmergencia(): boolean*. Retorna true si todos los sensores de la grilla están en riesgo.
- *hayNRiesgoCol(c, n: entero): boolean*. Retorna true si hay a lo sumo n sensores en riesgo en la columna c. Requiere 0 <= c < cantColumnas().
- *dosConsecutivosEmergenciaFila(f: entero): boolean*. Requiere 0 <= f < cantFilas(). Retorna true si la fila f contiene dos sensores en riesgo en posiciones consecutivas.

- *contarCoincidencias(gs:TableroSensores):entero*. Retorna la cantidad de sensores que, ocupando la misma posición en las dos grillas, tienen el mismo estado interno.
- *equals(t: TableroSensores):boolean*. Retorna true si y solo si el tablero que recibe el mensaje y el que pasa como parámetro tienen sensores equivalentes en exactamente las mismas posiciones.
- *clone():TableroSensores*. Implementa clone en profundidad. Requiere que todas las referencias del arreglo estén ligadas al momento de clonar.

Atención:

- ✓ Salvo para el método *clone()*, los recorridos no pueden asumir que todas las referencias de la grilla estén ligadas.
- ✓ En *contarCoincidencias(gs: TableroSensores): entero* las grillas pueden tener diferente número de filas y columnas, de modo que el recorrido puede ser parcial. Es decir, si una grilla es de 3x2 y otra de 2x4, se recorrerá 2x2
- ✓ En *equals(t: TableroSensores):boolean* del sí solo sí se infiere que si las grillas tienen diferente número de filas o de columnas debe retornar false.